

A Self-Hosted AI Feedback System for Formative Assessment: Implementation Guide Using n8n and Ollama

Harish Gautam ¹

¹ College of New Caledonia, Prince George, BC, Canada  Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Submitted: 07 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

This paper presents a complete implementation guide for building a self-hosted AI feedback system designed to support formative assessment in higher education. The system uses n8n (a visual workflow automation platform) combined with Ollama (a local large language model server) to provide students with detailed, rubric-aligned feedback on draft submissions. Students can request feedback on their drafts as many times as needed, typically receiving responses within 15-30 seconds, without any per-use costs or privacy concerns associated with cloud-based AI services. The implementation runs entirely on institutional servers using open-source components, with total costs under \$1,500 for hardware that can serve multiple courses and hundreds of students. We demonstrate the system through two real-world applications: providing formative feedback for complex essay exams in a customer relationship management course, and generating technical evaluations of student-built websites in a digital marketing course. All workflow files, configurations, and installation documentation are openly shared to enable community adoption and adaptation.

Repository: <https://github.com/hgautam4u/formative-feedback-system>

License: MIT

Statement of Need

Frequent, detailed feedback significantly improves student learning outcomes (Black & Wiliam, 1998; Hattie & Timperley, 2007), yet practical constraints often prevent instructors from providing the frequency and depth of feedback that research recommends (Shute, 2008). Faculty teaching large classes face difficult choices about how much time to invest in feedback versus other teaching responsibilities. Cloud-based AI services like ChatGPT or Claude offer potential solutions but introduce new problems: recurring subscription costs, privacy concerns with processing student data on third-party servers (Reidenberg & Schaub, 2018), and ethical questions about using AI for high-stakes grading (Selwyn, 2019).

This system addresses these challenges by focusing specifically on formative assessment rather than summative grading. Students receive AI-generated feedback on draft work to guide revision and improvement, but all final grades remain under complete instructor control. The self-hosted architecture ensures student data never leaves institutional servers, eliminating privacy concerns and compliance complications. After initial hardware investment (typically \$500-1,800), the system operates at zero marginal cost, making unlimited feedback requests sustainable even for large courses.

The need for such systems extends beyond individual instructors. Institutions increasingly seek alternatives to commercial cloud services that preserve institutional autonomy,

protect student privacy, and avoid vendor lock-in (Zawacki-Richter et al., 2019). Recent advances in open-source language models have made self-hosting more practical than many educators might expect, yet few comprehensive guides exist for deploying these systems in educational contexts (Holmes et al., 2019). This work fills that gap by providing complete documentation suitable for faculty and educational technologists without extensive programming backgrounds.

Software Description

The system architecture combines three main components. The n8n workflow engine provides visual, code-free workflow design, making the system accessible to educators without programming expertise. Users create workflows by connecting pre-built nodes through a drag-and-drop interface, with each node handling specific tasks like reading files, formatting text, or calling APIs. Ollama serves as the runtime environment for large language models, handling model loading, inference optimization, and resource management across different hardware configurations. Open-weights language models including Qwen2.5 (Yang et al., 2024), Gemma 2 (Gemma Team, 2024), and Llama 2 (Touvron et al., 2023) provide the underlying AI capabilities.

The typical workflow follows a pattern that adapts to different assessment types. Students submit draft work through a web form or learning management system integration. The system extracts text content, constructs a detailed prompt incorporating the course rubric and evaluation criteria, sends the request to the language model, parses the structured response, and generates a formatted feedback document. Processing time averages 15-30 seconds depending on submission length and hardware configuration. Students can repeat this process as many times as they wish as they revise their work, with each iteration providing updated feedback reflecting their changes, aligning with the “assessment for learning” philosophy (Wiliam, 2011).

Hardware requirements scale based on usage patterns. A repurposed laptop with 8 CPU cores and 32GB RAM can serve a single course with 30-50 students. Our implementation uses a former Dell XPS 15 laptop (Intel i7, 64GB RAM, 4GB GPU) running Ubuntu Linux, currently serving two courses with approximately 60 students total. Larger deployments might use dedicated servers with 16+ cores and 64GB RAM to support multiple courses or concurrent users. Optional GPU acceleration improves speed but is not required, as models run efficiently on CPU with appropriate optimization.

Model selection involves trade-offs between speed, quality, and resource requirements. We tested four models extensively:

- qwen2.5:7b (4.7GB): Best overall balance, excellent quality, 15-20 second responses
- qwen2.5:14b (9GB): Highest quality, slower but thorough, 25-35 second responses
- gemma2:9b (5.5GB): Good alternative, slightly less detailed feedback, 18-25 seconds
- llama3.1:8b (4.9GB): Solid baseline, adequate for most uses, 20-25 seconds

Based on extensive testing across different assignment types, we recommend qwen2.5:7b for most educational applications. It provides detailed, rubric-aligned feedback while running efficiently on modest hardware. Institutions with more powerful servers might prefer qwen2.5:14b for slightly higher quality, while resource-constrained deployments can use smaller models with acceptable results.

Key Features

Privacy preservation. All processing occurs on institutional servers. Student submissions never transmit to external services, simplifying FERPA compliance and eliminating third-party data processing concerns. Institutions maintain complete control over data retention, access, and deletion policies.

91 **Cost sustainability.** After initial hardware investment, operational costs consist only
92 of electricity (approximately \$10-15 monthly for a small server). Students can request
93 unlimited feedback without per-use charges, subscription fees, or API costs. This economic
94 model supports pedagogical best practices around frequent formative feedback without
95 ongoing budget implications.

96 **Pedagogical appropriateness.** The system explicitly supports formative assessment
97 (feedback for learning) rather than summative grading (final evaluation). Students
98 understand they receive AI-generated feedback to guide improvement, with final grades
99 remaining entirely under instructor control. This framing addresses ethical concerns about
100 algorithmic bias and preserves instructor expertise in high-stakes assessment.

101 **Accessibility for non-programmers.** The visual workflow interface requires no coding
102 knowledge. Educators familiar with basic computer use can create and modify workflows
103 through form-based configuration and drag-and-drop connections. Documentation includes
104 detailed screenshots and step-by-step instructions written for faculty without technical
105 backgrounds.

106 **Flexibility across disciplines.** The same infrastructure adapts to different assessment
107 types, disciplines, and rubric structures. Instructors customize workflows for their specific
108 needs by modifying prompt templates and evaluation criteria rather than rewriting code.
109 Our implementations demonstrate this versatility through two quite different use cases:
110 essay-based exams and technical website evaluation.

111 **Integration capabilities.** The system exposes API endpoints enabling integration with
112 learning management systems, custom web interfaces, or command-line tools. Basic
113 implementations use file upload and download, while more sophisticated deployments
114 might integrate directly with Canvas, Moodle, or institutional portals.

115 Implementation

116 Installation requires approximately 2-4 hours for someone comfortable following technical
117 documentation. The process involves five main steps: preparing a Linux server (Ubuntu
118 24.04 recommended), installing Docker and Docker Compose, deploying Ollama with
119 selected language models, deploying n8n, and importing pre-configured workflows. Our
120 repository includes a single-command installation script automating most setup tasks.

121 For users unfamiliar with Linux server administration, we provide detailed documentation
122 with screenshots for each step. Common troubleshooting scenarios are documented
123 based on testing across different hardware configurations. The installation guide assumes
124 basic familiarity with command-line interfaces but explains each command's purpose and
125 expected output.

126 Model configuration requires some optimization for educational use. Language models
127 typically default to creative, varied outputs appropriate for general conversation.
128 Educational feedback requires consistent, rubric-aligned responses. Our configuration files
129 reduce randomness (temperature 0.1), limit vocabulary to more precise terms (top_k 10,
130 top_p 0.3), and slightly penalize repetition. These parameters produce reliable, structured
131 feedback suitable for student learning rather than creative or unpredictable responses.

132 Workflow customization follows documented patterns. The repository includes two
133 complete workflow examples serving as templates. Educators can adapt these by modifying
134 rubric criteria, adjusting feedback length or tone, changing output formats, or adding
135 institution-specific integration points. The visual interface makes these modifications
136 accessible without programming expertise.

137 **Use Cases**

138 **Essay exam formative feedback.** In an upper-level customer relationship management
139 course, students complete a complex three-part midterm exam requiring analysis of
140 personal customer experiences, development of service recovery strategies, and proposal
141 of resource-constrained business improvements. The exam design deliberately requires
142 authentic personal examples to discourage AI-generated submissions. However, students
143 often struggle to assess whether draft answers meet rubric criteria.

144 We created a workflow accepting student draft submissions and evaluating them against
145 detailed criteria. Part 1 (35 points) assesses customer experience documentation,
146 relationship opportunity identification, communication behavior analysis, and technology
147 evaluation. Part 2 (35 points) covers service recovery scripting and video justification. Part
148 3 (30 points) evaluates strategic proposals under specified budget and time constraints.

149 The system generates personalized feedback quoting specific phrases from student
150 submissions and identifying strengths and improvement opportunities relative to rubric
151 criteria. For example, feedback might note “You described ‘felt embarrassed when called
152 back loudly’ which effectively captures emotional impact” while suggesting “consider
153 explicitly referencing ‘service encounter quality’ from Week 3 when discussing the return
154 interaction.”

155 Students averaged 2.1 feedback requests before final submission. Student comments
156 suggested the system helped them “understand what the rubric really meant” and reduced
157 anxiety about meeting requirements. Office hours shifted from rubric clarification toward
158 deeper conceptual discussions. While we have not conducted formal learning outcome
159 studies, instructor observation suggests final submissions showed better rubric alignment
160 compared to previous semesters without formative feedback access.

161 Processing averaged 22 seconds per submission using qwen2.5:7b on CPU-only mode (our
162 laptop lacks sufficient GPU memory for the full model). The system handled concurrent
163 requests from multiple students without performance issues. No technical failures occurred
164 during the semester.

165 **Website technical evaluation.** A digital marketing course requires students to build
166 WordPress websites demonstrating SEO optimization, keyword research, and landing page
167 design principles. Students struggle to self-assess technical elements like heading hierarchy,
168 meta description quality, or keyword placement. Traditional feedback required instructor
169 review of 35 websites, consuming 8-10 hours.

170 We developed a workflow analyzing websites against a technical checklist covering 15
171 criteria: proper H1 tag usage, meta description presence and length, image alt text,
172 keyword density, mobile responsiveness, page load speed, and other SEO fundamentals.
173 The workflow extracts website content, evaluates each criterion, assigns scores, and
174 generates improvement recommendations.

175 The system evaluated all 35 websites in approximately 12 minutes. Students received
176 specific, actionable feedback like “meta description current: 89 characters, should be
177 150-160 characters” or “keyword density too high at 4.2%, reduce to 1-2%.” I spot-checked
178 10 automated evaluations against my own manual review and found 90% agreement on
179 major technical issues, with discrepancies primarily in subjective quality judgments.

180 This workflow freed instructor time from technical checklist evaluation, enabling focus on
181 higher-level feedback about marketing strategy and content quality. Students appreciated
182 the immediate, specific technical guidance supporting their learning process.

Community and Sustainability

The repository provides complete implementation artifacts: Docker Compose configurations, workflow templates, model configuration files, installation scripts, and troubleshooting documentation. Community members can adopt these directly or adapt them for different educational contexts. We welcome contributions including additional workflow templates for different disciplines, learning management system integration modules, accessibility improvements, and model fine-tuning guides.

Beyond the code repository, we maintain supplementary resources including video walkthroughs of installation and configuration, frequently asked questions addressing common technical issues, and a discussion forum for community support. The open-source nature enables continuous improvement as community members share adaptations and enhancements.

Limitations and Future Directions

Current limitations include the technical knowledge required for initial setup, although detailed documentation reduces this barrier substantially. Hardware investment (\$500-1,800) may challenge some individuals or small institutions, though costs remain modest compared to multi-year cloud subscriptions. Model quality, while excellent for rubric-based feedback, remains somewhat below GPT-4 for nuanced writing evaluation, though improvements occur regularly as the open-source community releases better models.

Planned enhancements include streamlined installation scripts for Windows and macOS, pre-built plugins for Canvas and Moodle, mobile-friendly student interfaces, and instructor analytics dashboards. We invite the research community to conduct formal studies on student perceptions, learning outcomes, and equity implications of AI formative feedback systems.

Author Contribution

HG designed the system architecture, developed the workflows, conducted the implementations in two courses, and wrote the manuscript.

Acknowledgments

This work builds on contributions from the n8n, Ollama, and language model research communities. We thank students who provided feedback on system usability and effectiveness.

Black, P., & Wiliam, D. (1998). Assessment and classroom learning. *Assessment in Education: Principles, Policy & Practice*, 5(1), 7–74. <https://doi.org/10.1080/0969595980050102>

Gemma Team. (2024). Gemma 2: Improving open language models at a practical size. *arXiv Preprint arXiv:2408.00118*. <https://doi.org/10.48550/arXiv.2408.00118>

Hattie, J., & Timperley, H. (2007). The power of feedback. *Review of Educational Research*, 77(1), 81–112. <https://doi.org/10.3102/003465430298487>

Holmes, W., Bialik, M., & Fadel, C. (2019). *Artificial intelligence in education: Promises and implications for teaching and learning*. Center for Curriculum Redesign. ISBN: 978-1794296251

Reidenberg, J. R., & Schaub, F. (2018). Achieving big data privacy in education. *Theory and Research in Education*, 16(3), 263–279. <https://doi.org/10.1177/1477878518805308>

Selwyn, N. (2019). *Should robots replace teachers? AI and the future of education*. Polity Press. ISBN: 978-1509528967

- 228 Shute, V. J. (2008). Focus on formative feedback. *Review of Educational Research*, 78(1),
229 153–189. <https://doi.org/10.3102/0034654307313524>
- 230 Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N.,
231 Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M.,
232 Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., ... Scialom, T. (2023). Llama
233 2: Open foundation and fine-tuned chat models. *arXiv Preprint arXiv:2307.09288*.
234 <https://doi.org/10.48550/arXiv.2307.09288>
- 235 Wiliam, D. (2011). What is assessment for learning? *Studies in Educational Evaluation*,
236 37(1), 3–12. <https://doi.org/10.1016/j.stueduc.2011.03.001>
- 237 Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Li, C., Liu, D., Huang,
238 F., Dong, G., Wei, H., Tang, H., Zhang, J., Ma, J., Yang, J., Bai, J., Bai, J., Yang,
239 J., ... Zhu, Z. (2024). Qwen2.5 technical report. *arXiv Preprint arXiv:2412.15115*.
240 <https://doi.org/10.48550/arXiv.2412.15115>
- 241 Zawacki-Richter, O., Marín, V. I., Bond, M., & Gouverneur, F. (2019). Systematic review
242 of research on artificial intelligence applications in higher education. *International*
243 *Journal of Educational Technology in Higher Education*, 16(1), 1–27. <https://doi.org/10.1186/s41239-019-0171-0>
244